

DESAFIOS DE APROFUNDAMENTO

Programação Backend - JavaScript / Node.js

Atividade individual para ampliação e consolidação dos conteúdos

Aluno(a)	
Turma	Castello
Data	
Professor	

Objetivo da atividade

Esta atividade foi preparada para aprofundar os conteúdos já estudados em JavaScript. As três situações-problema exigem integração entre diferentes recursos da linguagem e maior autonomia na organização da solução.

Serão utilizados apenas conteúdos já trabalhados: variáveis, operadores, entrada de dados com `readline-sync`, estruturas condicionais, laços de repetição, arrays, objetos, funções com parâmetros e retorno, além de modularização com `require()` e `module.exports`.

Orientações gerais

- ☐ Leia toda a situação-problema antes de começar a programar.
- ☐ Planeje quais dados precisam ser armazenados e quais estruturas serão necessárias.
- ☐ Utilize `readline-sync` sempre que o enunciado solicitar dados ao usuário.
- ☐ Crie mensagens claras no terminal para facilitar a utilização do programa.
- ☐ Teste os programas com valores diferentes, inclusive valores próximos aos limites das condições.
- ☐ Evite repetir código quando uma função ou um laço puder resolver o problema.
- ☐ Nas funções, utilize parâmetros para receber dados e `return` para devolver resultados.
- ☐ No desafio modular, mantenha as regras de negócio no módulo e deixe o `app.js` responsável pelas entradas, chamadas e relatório.

SITUAÇÃO-PROBLEMA 1 - Monitoramento de produção por turno

Contexto

Uma indústria deseja acompanhar o desempenho de uma máquina durante um turno de trabalho. O operador registra, ao final de cada hora, quantas peças foram produzidas. A supervisão precisa de um relatório simples que permita verificar o desempenho do turno e identificar se a meta de produção foi alcançada.

Desafio

Desenvolva um programa que solicite o nome da máquina e registre a produção de 5 horas de trabalho. Cada valor de produção deverá ser armazenado em um array. Ao final, o sistema deverá apresentar um relatório do turno.

Requisitos mínimos

- ☐ Solicitar o nome da máquina.
- ☐ Usar um laço de repetição para receber a produção de cada uma das 5 horas.
- ☐ Armazenar todas as produções em um array.
- ☐ Calcular a produção total do turno.
- ☐ Calcular a média de produção por hora.
- ☐ Identificar a maior produção registrada entre as 5 horas.
- ☐ Considerar uma meta de 500 peças no turno.
- ☐ Exibir META ATINGIDA quando o total for maior ou igual a 500; caso contrário, exibir META NÃO ATINGIDA.
- ☐ Criar uma função calcularMedia(total, quantidadeHoras) e utilizar seu retorno no relatório.

Arquivo(s) para entrega	desafio01_producao.js
Desafio adicional	Além do relatório final, mostre também todas as produções registradas, numeradas por hora.

SITUAÇÃO-PROBLEMA 2 - Sistema de estoque e reposição de componentes**Contexto**

O almoxarifado de uma indústria precisa acompanhar os componentes utilizados no processo produtivo. Cada componente possui uma quantidade disponível, um estoque mínimo e um valor unitário. A equipe precisa identificar rapidamente os itens que devem ser repostos e conhecer o valor financeiro armazenado em estoque.

Desafio

Desenvolva um programa para cadastrar 5 componentes. Cada componente deverá ser representado por um objeto, e todos os objetos deverão ser armazenados em um array.

Requisitos mínimos

- ☐ Criar um array vazio para armazenar os componentes.
- ☐ Cadastrar 5 componentes usando um laço de repetição.
- ☐ Para cada componente, solicitar nome, quantidade atual, estoque mínimo e valor unitário.
- ☐ Criar um objeto para cada componente e adicioná-lo ao array com push().
- ☐ Percorrer o array após o cadastro.
- ☐ Se a quantidade atual for menor que o estoque mínimo, exibir REPOR ESTOQUE; caso contrário, ESTOQUE OK.
- ☐ Criar a função calcularValorEstoque(quantidade, valorUnitario).
- ☐ Calcular o valor financeiro de cada componente em estoque.
- ☐ Calcular o valor total de todos os componentes armazenados.
- ☐ Exibir um relatório com nome, quantidade, estoque mínimo, valor unitário, valor em estoque e situação.

Arquivo(s) para entrega	desafio02_estoque.js
Desafio adicional	Ao final, informe também quantos componentes precisam de reposição.

SITUAÇÃO-PROBLEMA 3 - Sistema modular de manutenção industrial**Contexto**

Uma empresa presta serviços de manutenção em diferentes tipos de máquinas. Como os serviços podem exigir profissionais e níveis de especialização diferentes, o valor da hora técnica não é fixo e deve ser informado no momento do atendimento. A empresa quer separar as regras de cálculo do restante do programa para facilitar futuras alterações.

Desafio

Desenvolva a solução utilizando dois arquivos JavaScript. As funções de cálculo e classificação deverão ficar em um módulo separado, enquanto o programa principal será responsável por receber os dados do atendimento e apresentar o relatório.

Requisitos mínimos

- ☐ Criar uma pasta chamada desafio03_manutencao.
- ☐ Dentro dela, criar funcoesManutencao.js e app.js.
- ☐ Em funcoesManutencao.js, criar calcularMaoDeObra(horas, valorHora), retornando o custo da mão de obra.
- ☐ Criar calcularTotal(valorPecas, horas, valorHora), retornando peças + mão de obra.
- ☐ Criar classificarManutencao(total).
- ☐ Classificação: até R\$ 500,00 = MANUTENÇÃO DE BAIXO CUSTO.
- ☐ Acima de R\$ 500,00 e até R\$ 1.500,00 = MANUTENÇÃO DE MÉDIO CUSTO.
- ☐ Acima de R\$ 1.500,00 = MANUTENÇÃO DE ALTO CUSTO.
- ☐ Exportar as funções usando module.exports.
- ☐ No app.js, importar readline-sync e o módulo com require().
- ☐ Solicitar nome da máquina, setor, valor das peças, quantidade de horas e valor da hora técnica.
- ☐ Chamar as funções do módulo utilizando os dados informados pelo usuário.
- ☐ Exibir relatório contendo máquina, setor, valor das peças, horas, valor da hora, mão de obra, custo total e classificação.

Arquivo(s) para entrega	desafio03_manutencao/app.js + desafio03_manutencao/funcoesManutencao.js
Desafio adicional	Teste o programa com pelo menos três cenários diferentes, de forma que cada faixa de classificação apareça pelo menos uma vez.

Checklist final

- ☐ Todos os programas foram executados e testados.
- ☐ As entradas solicitadas pelo enunciado foram implementadas.
- ☐ As saídas apresentam todas as informações pedidas.
- ☐ As funções utilizam parâmetros e return quando solicitado.
- ☐ Arrays e objetos foram utilizados de acordo com a situação-problema.
- ☐ No desafio 3, as funções foram separadas corretamente do app.js.
- ☐ Os três cenários de teste do desafio 3 foram executados.

Orientação final: antes de considerar uma situação concluída, faça pelo menos um teste com valores diferentes dos que você utilizou durante o desenvolvimento. O objetivo é verificar se a lógica continua funcionando em outros cenários.